

A1 - Assignment 2

THEORY

Q1)

Language:

g_i represent light being green at instant i
 y_i represent light being yellow at instant i
 r_i represent light being red at instant i

Propositions:

$$\textcircled{1} (g_i \leftrightarrow (\neg y_i \wedge \neg r_i)) \wedge (y_i \leftrightarrow (\neg g_i \wedge \neg r_i)) \\ \wedge (r_i \leftrightarrow (\neg g_i \wedge \neg y_i))$$

$$\textcircled{2} (g_{i-1} \rightarrow (g_i \vee y_i)) \wedge (y_{i-1} \rightarrow (y_i \vee r_i)) \\ \wedge (r_{i-1} \rightarrow (r_i \vee g_i))$$

$$\textcircled{3} (g_{i-3} \wedge g_{i-2} \wedge g_{i-1} \rightarrow \neg g_i) \\ (y_{i-3} \wedge y_{i-2} \wedge y_{i-1} \rightarrow \neg y_i) \\ (r_{i-3} \wedge r_{i-2} \wedge r_{i-1} \rightarrow \neg r_i)$$

Q2 Let

- Binary predicate, $\text{edge}(n, m)$: node n is connected to node m
- Binary predicate, $\text{color}(n, c)$: node n has color c
- constants: yellow, green, red

Axioms:

1.

$$\forall n \forall m \forall c (\text{edge}(n, m) \wedge \text{color}(n, c) \rightarrow \neg \text{color}(m, c))$$

2.

$$\exists n_1, \exists n_2 (\text{color}(n_1, \text{yellow}) \wedge \text{color}(n_2, \text{yellow}) \wedge$$

$$(n_1 \neq n_2) \wedge \forall m (m \neq n_1 \wedge m \neq n_2$$

$$\rightarrow \neg \text{color}(m, \text{yellow})))$$

$$\rightarrow \neg \text{color}(m, \text{yellow})))$$

$$3. \forall n (\text{color}(n, \text{red}) \rightarrow$$

$$(\exists n_1 (\text{edge}(n, n_1) \wedge \text{color}(n_1, \text{green})) \vee$$

$$\exists n_1, n_2 (\text{edge}(n, n_1) \wedge (\text{edge}(n_1, n_2) \wedge \text{color}(n_2, \text{green})) \vee$$

$$\exists n_1, n_2, n_3 (\text{edge}(n, n_1) \wedge \text{edge}(n_1, n_2) \wedge \text{edge}(n_2, n_3) \wedge \text{color}(n_3, \text{green})) \vee$$

$$\exists n_1, n_2, n_3, n_4 (\text{edge}(n, n_1) \wedge \text{edge}(n_1, n_2) \wedge \text{edge}(n_2, n_3) \wedge \text{edge}(n_3, n_4) \wedge \text{color}(n_4, \text{green}))$$

))

4.

$$\forall x \exists n \text{ color}(n, x)$$

5.

$$\forall x \exists n \text{ color}(n, x) \wedge$$

$$\forall n \exists x \text{ color}(n, x) \wedge$$

$$\forall n \forall x \text{ color}(n, x) \rightarrow \neg \exists y (y \neq x$$

$$\wedge \text{color}(n, y)) \wedge$$

$$\forall n \forall m \forall x (n \neq m \wedge \text{color}(n, x)$$

$$\wedge \text{color}(m, x) \rightarrow$$

$$(edge(n, m) \bigwedge_{i=1}^{|N|} \exists n_1, \dots, n_i (edge(n, n_1^{(i)}) \bigwedge_{j=1}^{i-1} edge(x_j, x_{j+1}) \wedge edge(n_i, m))))$$

Q3 Let

$\text{dolphin}(x)$ represent x is a dolphin, D
 $\text{read}(x)$ represent x can read, R
 $\text{intelligent}(x)$ represent x is intelligent, I
 $\text{literate}(x)$ represent x is literate, L

FOL

- whoever can read is literate

$$\forall x (\text{read}(x) \rightarrow \text{literate}(x))$$

$$PL: R \rightarrow L \quad \equiv \quad \neg R \vee L$$

- dolphins, unfortunately, are not literate

$$\forall x (\text{dolphin}(x) \rightarrow \neg \text{literate}(x))$$

$$PL: D \rightarrow \neg L \quad \equiv \quad \neg D \vee \neg L$$

- Some dolphins are intelligent

$$\exists x (\text{dolphin}(x) \wedge \text{intelligent}(x))$$

$$PL: D \wedge I$$

- Some who are intelligent cannot read

$\exists$

$$\exists x (\text{intelligent}(x) \wedge \neg \text{Read}(x))$$

$$PL: I \wedge \neg R$$

- There exist a dolphin who is both ~~it~~ intelligent and can read, but for every intelligent dolphin, if it can read, it must be that it is not literate

$$PL: D \wedge I \wedge R \wedge (I \wedge R \rightarrow \neg L)$$

$$\forall y \exists x (\text{dolphin}(x) \wedge \text{intelligent}(x) \wedge \text{read}(x))$$

$$\wedge$$
$$(\text{dolphin}(y) \wedge \text{intelligent}(y) \wedge \text{read}(y))$$
$$\rightarrow \neg \text{literate}(y))$$

Satisfiability of 4th Statement

- Lets Assume

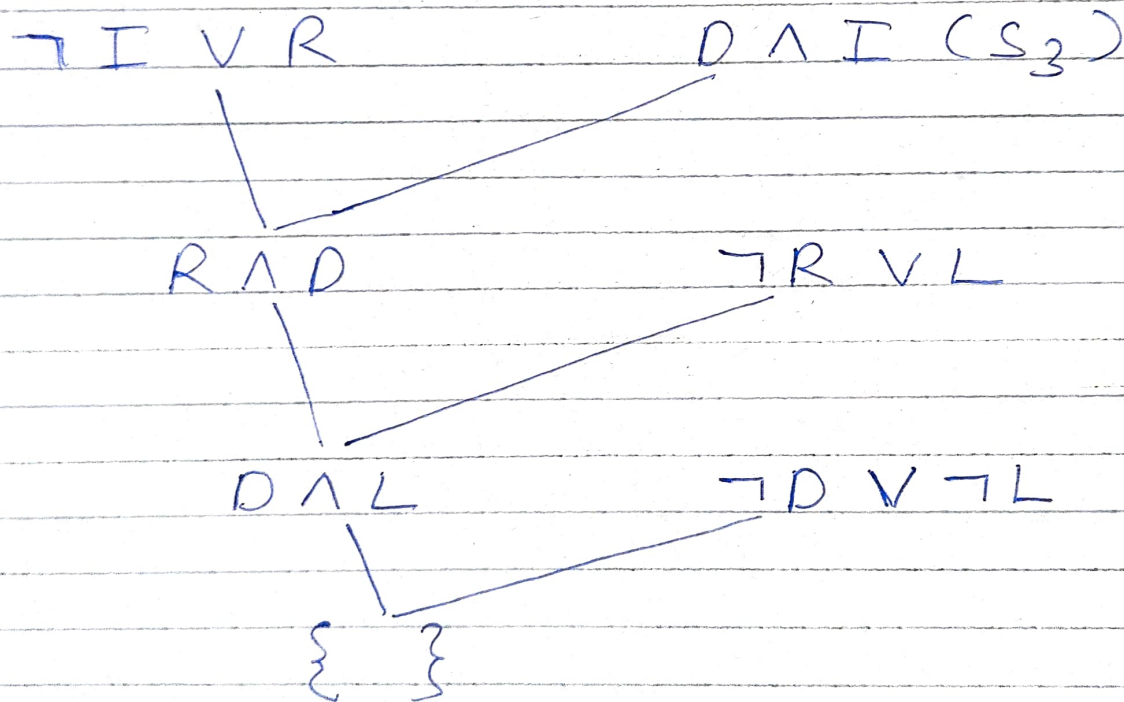
~~$\neg \exists x$~~

$$\neg (\exists x (\text{intelligent}(x) \wedge \neg \text{Read}(x)))$$

$$\equiv \forall x (\neg \text{intelligent}(x) \vee \text{read}(x))$$

$$\equiv \forall x \text{ intelligent}(x) \rightarrow \text{read}(x)$$

$\hookrightarrow$ all intelligent can read



Since we arrived at contradiction.
Since we arrived at an empty clause,
it is a contradiction.
 $\therefore$ Statement 4 is true and is satisfiable.

Satisfiability of 5th Statement

~~Not to Assume~~
Consider

$$\neg (D(x) \wedge I(x) \wedge R(x))$$

$$\neg D(x) \vee \neg I(x) \vee \neg R(x) \quad I(x)$$

$$\neg D(x) \vee \neg R(x) \quad D(x)$$

$$\neg R(x)$$

Cannot reach to an empty clause
 $\therefore$ 5th Statement is unsatisfiable

Statement 5's requirement

- dolphin ~~can read~~ should be both intelligent and can read also if it can read it cannot be intelligent

$$D \wedge I \wedge R \rightarrow \neg L$$

But Statement 1 says: All those who can read ~~cannot~~ be literate

$$R \rightarrow L$$

Thus contradiction.

Computational

GTFS static data from Delhi's Open Transit Data

1. Data Loading and Knowledge Base Creation

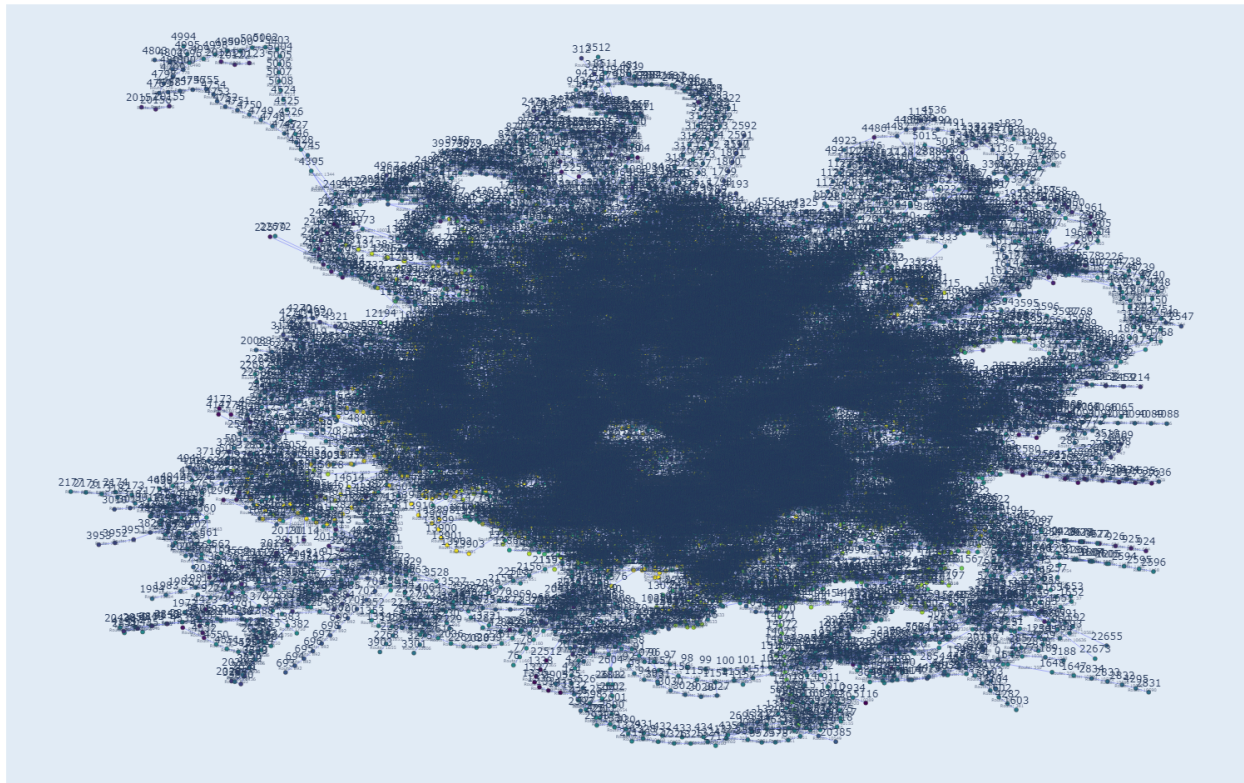
Busiest_routes: [(5722, 318), (5721, 318), (674, 313), (593, 311), (5254, 272)]

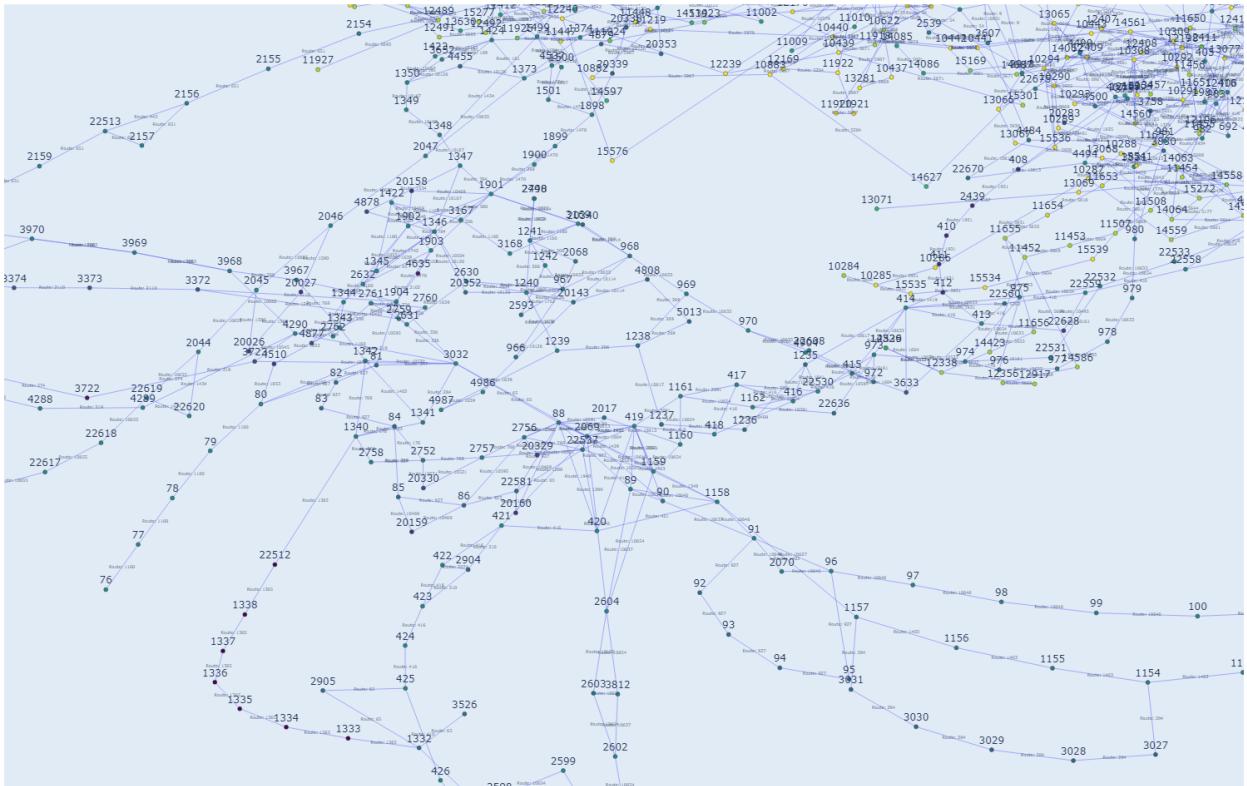
Most_frequent_stops: [(10225, 4115), (10221, 4049), (149, 3998), (488, 3996), (233, 3787)]

Busiest_stops: [(488, 102), (10225, 101), (149, 99), (233, 95), (10221, 86)]

Stops_with_one_direct_route: [((10060, 10221), 5607), ((10221, 11475), 5607), ((149, 634), 1929), ((10060, 10775), 5376), ((10221, 11476), 5607)]

Visualization





2. Reasoning

- a) We sample 100 pairs directly from all possible unique stop pairs in the dataset to better represent the entire data range while limiting computational load. Each sampled pair (start, end) is evaluated using the functions `direct_route_brute_force` and `query_direct_routes`, measuring both execution time (in milliseconds) and peak memory usage (in megabytes). After processing these sampled pairs, we calculate the average time and memory for each function.

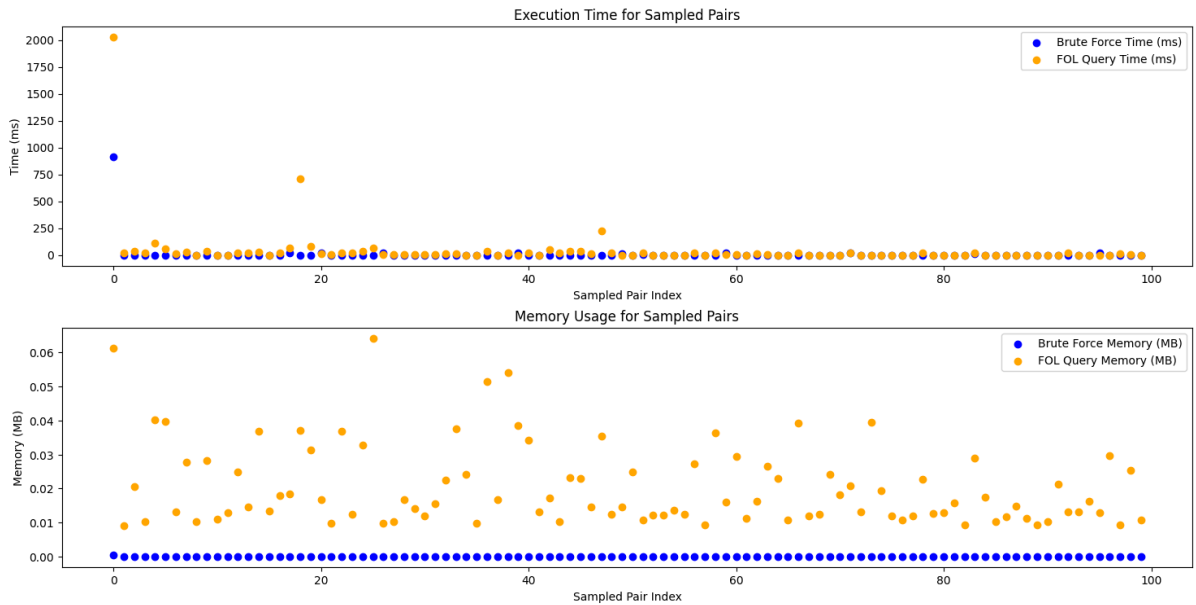
-----REASONING EVALUATION RESULTS-----

Brute force average time (ms): 10.492749214172363

Brute force average memory (MB): 0.0001120758056640625

FOL query average time (ms): 40.50917387008667

FOL query average memory (MB): 0.020508108139038087



b)

```
python "f:\AI\Assignment 2\A2_test.py"
Terms initialized: DirectRoute, RouteHasStop, OptimalRoute
Test direct_route_brute_force (2573, 1177): Pass
Test direct_route_brute_force (2001, 2005): Pass
Test query_direct_routes (2573, 1177): Pass
Test query_direct_routes (2001, 2005): Pass
```

In the **brute-force approach** for finding direct routes (`direct_route_brute_force` function), the intermediate steps are as follows:

1. **Iterate Through Routes:** The function iterates over all routes in the `route_to_stops` dictionary, where the key is the `route_id` and the value is a list of stops (`route_stops`).
2. **Check if Both Stops Are Present:** For each route, the function checks if both the `start_stop` and `end_stop` are present in the list `route_stops`. This is done using the condition `if ((start_stop in route_stops) and (end_stop in route_stops))`.
3. **Append the Route ID:** If both stops are present in the current route, the route ID (`route_id`) is added to the `direct_routes` list.
4. **Return the Result:** After iterating over all routes, the function returns the `direct_routes` list, which contains the IDs of all routes that directly connect the two given stops.

The brute-force method involves checking every route and verifying the presence of both stops in each route, which can be computationally expensive if the dataset of routes and stops is large.

In the **Datalog-based approach** for finding direct routes (`query_direct_routes` function), the reasoning steps are as follows:

1. **Define Datalog Predicates:**

- The predicates `RouteHasStop`, `DirectRoute`, and `OptimalRoute` are defined to represent relationships between routes, stops, and optimal paths between two stops.
- `DirectRoute(R, X, Y)` is defined as true if a route `R` has both stop `X` and stop `Y`.

2. **Initialize the Knowledge Base:**

- The function `initialize_datalog()` clears the previous terms and initializes the predicates in Datalog.
- The `create_kb()` function is called to populate the knowledge base with predefined rules.

3. **Add Route Data:**

- The `add_route_data(route_to_stops)` function adds data about the routes and their corresponding stops to the Datalog knowledge base using the `RouteHasStop` predicate.

4. **Query for Direct Routes:**

- The query `DirectRoute(R, start, end)` asks the knowledge base whether there is a route `R` that directly connects `start` and `end`.
- The `ask()` method returns a list of tuples containing route IDs that satisfy the query.

5. **Return the Result:**

- The route IDs are extracted from the result and sorted before returning as the final list of direct routes.

The Datalog-based approach utilizes a declarative reasoning process to infer the relationships between routes and stops using the rules defined in the knowledge base. The underlying database query engine (via the `ask()` method) handles the search for valid routes.

c) Let n is the number of routes, m is the number of stops per route, and q is the number of rules or queries.

- **Brute-Force Approach:** The time complexity is proportional to the number of routes and the number of stops per route $O(n \cdot m)$, which can become very large as the number of routes or stops increases.

- **Datalog-Based Approach:** The Datalog approach also scales with $O(n \cdot m) + O(q)$ for adding the route data, but its query execution time (depending on the complexity of the rules) is typically constant. If more complex rules are added, the query complexity could increase, but in practice, the reasoning process often leverages optimized search techniques.

Overall, both methods have similar time complexities in the worst case for adding and querying route data. However, the Datalog-based approach may provide more flexibility in reasoning and could optimize queries as the knowledge base grows, while the brute-force approach is more straightforward but less scalable.

3. Planning

- a) We used test inputs to find calculate the averages

-----PLANNING EVALUATION RESULTS-----

Forward chaining average time (ms): 8.322477340698242

Forward chaining average memory (MB): 0.09325599670410156

Backward chaining average time (ms): 8.247852325439453

Backward chaining average memory (MB): 0.07402420043945312

- b)

```
Test forward_chaining (22540, 2573, 4686, 1): Pass
Test forward_chaining (951, 340, 300, 1): Pass
Test backward_chaining (22540, 2573, 4686, 1): Pass
Test backward_chaining (951, 340, 300, 1): Pass
```

In both forward and backward chaining, the reasoning process begins by querying the knowledge base to identify potential routes between the starting and ending stops. In forward chaining, the system queries for routes that start from the given start stop, pass through a specified intermediate stop, and end at the destination stop. Once the relevant routes are identified, the results are sorted and returned. In backward chaining, the process is reversed: the query starts from the goal (end stop) and works backward to find routes that can lead to the start stop via an intermediate stop. After querying, the routes are reversed to maintain the correct order and then sorted. Both approaches involve querying, sorting, and returning results, but the backward chaining adds an extra step of reversing the routes, which is the main difference in their reasoning processes.

- c) Forward and backward chaining are both reasoning techniques used to find optimal routes, but they differ in the direction of reasoning. In forward chaining, the system starts from a known fact (the starting stop) and moves forward through the rules to infer the result (optimal routes), considering possible transfers. In backward chaining, the reasoning starts from the goal (the ending stop) and works backward, identifying intermediate steps (transfers) to reach the goal. While both approaches involve querying the knowledge base and sorting the results, backward chaining requires an additional step of reversing the route order after the query and before sorting.

4. Bonus

```
Test pddl_planning (22540, 2573, 4686, 1): Pass
Test pddl_planning (951, 340, 300, 1): Pass
Test bfs_route_planner_optimized (22540, 2573, 10, 3): Pass
Test bfs_route_planner_optimized (4012, 4013, 10, 3): Pass
```

-----BONUS EVALUATION RESULTS-----

PDDL average time (ms): 37.75456428527832

PDDL average memory (MB): 8.322477340698242

BFS average time (ms): 801.1221885681152

BFS average memory (MB): 48.247852325439453

Yes, all algorithms (forward chaining, backward chaining, and PDDL) produce the same optimal route.